SpatFD v2: salvedades de la implementación

Brief técnico del *pipeline* del paquete

Octubre de 2026

Este documento resume las principales salvedades de la implementación actual de SpatFD v2. No discute los métodos estadísticos en sí, sino las decisiones del *pipeline* del paquete: cómo se suaviza, cómo se hace el análisis de componentes principales funcionales, cómo se traen los modelos de variograma y, sobre todo, cómo se predice. Cada punto indica qué hace hoy el código y por qué conviene tenerlo presente.

1. Flujo del paquete

El flujo completo tiene cuatro pasos:

1. `spatfd()`: suavizamiento de las curvas y análisis de componentes principales funcionales (FPCA).
2. `togstat()`: exportación de los *scores* para estimar y ajustar variogramas por fuera del paquete.
3. `add_model()`: traducción, compilación y registro de los modelos ajustados dentro del objeto.
4. `predict()`: predicción de las curvas en sitios nuevos o sobre un mapa, con los métodos `scores` y `lambda`.

La predicción ya no depende de `gstat`: los sistemas se resuelven de forma matricial dentro del paquete. Tanto el método `scores` como el método `lambda` reproducen los resultados de la versión CRAN (0.0.1) con diferencias relativas del orden de 10^{-16}.

2. Suavizamiento

- **Operador de penalización.** Se define con `vec2Lfd(c(1, ncol(dat)))`, heredado de la versión CRAN. Con n estaciones el operador es

$$L\chi = D^2\chi + n\,D\chi + \chi,$$

  de modo que la penalización depende del número de estaciones. No hay una justificación evidente para ello y cambia el suavizamiento cuando se agregan o quitan sitios.

- **Parámetros fijos.** El parámetro de suavizamiento `lambda` y el número de bases `nbasis` los elige el usuario y son los mismos para todas las curvas. No hay selección automática, por ejemplo por validación cruzada generalizada.

- **Datos faltantes.** No se revisan valores faltantes en los datos; la versión CRAN sí lo hacía.

3. Componentes principales funcionales

- **Media constante en el espacio.** La función media $\mu(t)$ es el promedio simple de las curvas de las estaciones, sin considerar su ubicación. Todo lo que sigue supone que la media no cambia en el espacio.
- **Dependencia del número de componentes.** Los *scores* cambian ligeramente, en el cuarto decimal, según cuántos componentes se pidan, por la integración numérica de `fda`. Por eso, cuando se selecciona el número de componentes por varianza explicada, el análisis se repite en lugar de recortarse.
- **Un FPCA por variable.** Cada variable funcional tiene su propio análisis; no hay un análisis conjunto entre variables.

4. Modelamiento de los variogramas

- **Pocas estaciones.** Con pocas estaciones (13 para PM10 en los datos de México) los variogramas empíricos son muy inestables. En el ejemplo de demostración el ajuste de los variogramas de PM10 no converge.
- **Meseta frente a valor propio.** Nada obliga a que la meseta del modelo ajustado se parezca al valor propio del componente, que es la varianza empírica del *score*. La diferencia entre ambos afecta directamente la varianza de predicción.
- **Traducción desde gstat.** `toSpatFD()` solo conserva el modelo, el rango, la meseta parcial y el parámetro `kappa`. La anisotropía se descarta sin aviso, y un modelo `Lin` con rango se traduce como lineal no acotado.
- **Modelos que no coinciden con gstat.** Seis modelos espejo (`circular`, `bessel`, `pentamodel`, `logarithmic`, `spline` y `legendre`) no reproducen a `gstat`. Predecir con ellos da resultados distintos sin aviso.
- **Un solo traductor.** Solo se pueden traer modelos ajustados con `gstat`.

5. Predicción

5.1. Aspectos comunes a los dos métodos

- **Kriging simple con media cero.** Las covarianzas se obtienen de los variogramas como $C(h) = C(0) - \gamma(h)$, con $C(0)$ la suma de las mesetas parciales. Esto descansa en que los *scores* sean estacionarios y centrados.
- **Modelos sin meseta.** Los modelos `linear`, `power` y `logarithmic` no tienen covarianza y no se pueden usar hasta que exista kriging ordinario.
- **Distancias euclidianas.** Las distancias se calculan sobre las coordenadas tal como vienen. Sirven para coordenadas proyectadas, no para longitud y latitud.
- **Vecindario global.** Se factoriza una matriz densa con todos los sitios observados, lo que escala mal cuando hay muchos sitios. No hay kriging por vecindarios.
- **Validez del modelo.** Solo se detecta un modelo inválido si la factorización de Cholesky falla. Un modelo que no sea válido puede pasar sin error.
- **Efecto pepita.** Se trata como en `gstat`: el variograma vale cero en el origen y la predicción en un sitio observado devuelve el valor observado.

5.2. Método `scores`

Cada *score* se predice por kriging simple y la curva se reconstruye:

$$\hat{f}_k(s_0) = c_0^T \, \Sigma^{-1} f, \qquad \hat{\chi}_{s_0}(t) = \mu(t) + \sum_{k=1}^{K} \hat{f}_k(s_0) \, \xi_k(t).$$

- **Independencia entre scores.** Los *scores* de una misma variable se tratan como independientes salvo que el modelo traiga variogramas cruzados. El FPCA solo garantiza que no estén correlacionados en un mismo sitio, no a distancia.
- **Truncamiento.** La curva se reconstruye con los K componentes retenidos, y la varianza reportada, $\sum_k \sigma_k^2(s_0)$, ignora la variabilidad de los componentes que quedaron por fuera.
- **Un solo sistema.** Los casos univariado y multivariado pasan por el mismo *solver* por bloques. Los *scores* que no están conectados por variogramas cruzados se resuelven por aparte.

5.3. Método `lambda`

Se implementó igual que en la versión CRAN, por decisión explícita, para permitir la comparación directa:

$$\hat{\chi}_{s_0}(t) = \sum_{i=1}^{n} \lambda_i \, \chi_{s_i}(t), \qquad \Omega \, \lambda = c, \qquad \Omega = \sum_{k=1}^{K} \Omega_k, \quad c = \sum_{k=1}^{K} c_k,$$

con varianza de predicción

$$\sum_{k=1}^{K} \left(\eta_k - 2 \, c_k^T \lambda + \lambda^T \Omega_k \lambda \right),$$

donde η_k es el valor propio del componente k.

- **Curvas sin centrar.** Los pesos son de kriging simple y no suman uno, pero se aplican a las curvas observadas sin restarles la media. La predicción se encoge hacia cero y no hacia la media: en PM10 las curvas predichas quedan muy por debajo de las observadas. La versión coherente sería

  $$\hat{\chi}_{s_0}(t) = \mu(t) + \sum_{i=1}^{n} \lambda_i \big(\chi_{s_i}(t) - \mu(t) \big),$$

  y el lugar donde hacer el cambio está marcado en el código de `predict_lambda()`.
- **Varianza con el valor propio.** La varianza mezcla el valor propio η_k con las covarianzas del modelo ajustado. Si la meseta y el valor propio difieren mucho puede salir negativa, y no hay aviso.
- **No comparable con `scores`.** Por las dos razones anteriores, ni las curvas ni las varianzas del método `lambda` son comparables hoy con las del método `scores`.
- **Solo univariado.** Si el modelo tiene variogramas cruzados o se pide más de una variable, la función se detiene con un error.

6. Objeto y registro de modelos

- **Registro en el sitio.** `add_model()` modifica el objeto sin reasignarlo. Una copia hecha con `<-` comparte los modelos del objeto original.
- **Identificadores de variogramas cruzados.** Se arman uniendo los nombres de los *scores* con un punto. Nombres de variable que contengan puntos pueden volverse ambiguos.
- **Mapas.** Al predecir sobre un mapa no se revisa ni se transforma el sistema de coordenadas: debe ser el mismo de las estaciones. El mapa de predicción muestra un solo instante.

7. Pendientes

- Validación cruzada dejando un sitio por fuera, que es la herramienta prevista para comparar modelos y métodos.
- Kriging ordinario, necesario para los modelos sin meseta.
- Método `lambda` centrado y su extensión multivariada.
- Corregir los seis modelos espejo que no coinciden con `gstat`.